AQDC-BBR: An Active Queue Debt Compensation Mechanism over BBR for Reducing Tail Latency in Video Streaming

Ziyi Li*, JiaoZhang*, Yuqing Wang*, Peisheng Guo*, Jiafeng Jiang*,
*Beijing University of Posts and Telecommunications, Beijing, China

2025 Thirteenth International Conference on Advanced Cloud and Big Data (CBD) | 979-8-3315-7966-1/25/$31.00 ©2025 IEEE | DOI: 10.1109/CBD69312.2025.00020

***Abstract*—Modern model-based congestion control protocols such as BBR achieve high throughput by accurately estimating bottleneck bandwidth and RTT. However, during ProbeBW phases, bandwidth ramp-up can introduce transient queues that accumulate over successive cycles, leading to tail latency increase and degraded performance for delay-sensitive applications like video streaming. In this paper, we propose Active Queue Debt Compensation (AQDC-BBR), a lightweight extension to BBR that explicitly models the "queue debt" incurred during bandwidth probing and compensates it in subsequent phases. We evaluated AQDC-BBR in both ns-3 simulations and Mininet video streaming experiments.The results show that AQDC-BBR reduces tail latency by 26% and 25% compared to Copa and PCC, respectively, and achieves a reduction of more than 25% compared to BBR and its variants, while maintaining stable high throughput.**

***Index Terms*—Congestion Control, BBR, Video Streaming, Low Latency, Active Queue Debt Compensation**

I. Introduction

Modern interactive applications, such as real-time video conferencing, live streaming, and cloud gaming [1] [2], place stringent demands on congestion control algorithms (CCAs). These applications are highly sensitive to queuing delay, tail latency, and short-term throughput fluctuations, requiring transport protocols that can adapt quickly to dynamic network conditions without overfilling buffers or introducing delay spikes.

Traditional loss-based CCAs, such as TCP Reno and Cubic [3], rely on packet loss to infer congestion. While effective for maintaining high throughput, these approaches tend to build large queues and suffer from high latency. To mitigate these issues, several delay-based and hybrid congestion control algorithms have been proposed, including Vegas [6] and Copa [7], which attempt to control queueing delay more proactively. However, these approaches can sacrifice throughput under varying network conditions. More recent efforts have focused on model-driven CCAs, such as PCC [8] and Copa, which aim to strike a better balance between throughput and latency. However, many of these designs either converge slowly, misinterpret queue signals, or require parameter tuning that undermines their performance across networks [18].

This work is supported by the National Key RD Program of China under Grant No.2022YFB2901700

BBR represents a significant shift in congestion control design. Rather than reacting to loss or delay, BBR [4] estimates the bottleneck bandwidth and minimum RTT, tries to actively operate at Kleinrock's optimal operating point [5] to maximize throughput without incurring standing queues at a bottleneck link. However, BBRv1's periodic probing mechanism often injects bursts of traffic that exceed the path's instantaneous capacity, creating temporary queues. These residual queues may not be fully drained, especially when bandwidth estimates are overly optimistic. Over time, this leads to a drift in minRTT measurements, degrading the model's accuracy [9] [10] [11]. To address these limitations, BBRv2 [12] incorporates loss and ECN feedback [13], introduces a lower congestion window cap, and adopts more conservative probing [14] [15]. Building on this, BBRv3 [16] [17] was recently released. It retains the core model-driven design of its predecessors but refines pacing gains and improves interactions with bottleneck queues.

However, despite these refinements, all BBR variants still rely on periodic bandwidth probing as a fundamental mechanism. This probing inevitably introduces queue buildup as a side effect. In general-purpose scenarios, the resulting delay may be tolerable, but in real-time video streaming, these transient queues can trigger delay spikes that lead to playback stalls, rebuffering events, and dropped frames, degrading the user experience significantly.

To address this issue, we propose AQDC-BBR (Active Queue Debt Compensation BBR), a simple but effective extension to BBR that explicitly models the queuing pressure introduced in each probing cycle, referred to as queue debt, and incorporates a mechanism to proactively repay this pressure in subsequent phases. AQDC-BBR retains the original probing structure of BBR but augments it with a lightweight accounting system that tracks the difference between successive bandwidth estimates and compensates for the excess inflight data accordingly.

We implement the AQDC-BBR algorithm in both NS-3 [19] and mininet [20], and the simulation and experimental results demonstrate that: 1) AQDC-BBR exhibits robustness across diverse network scenarios; 2) Compared with Copa and PCC, AQDC-BBR reduces tail latency by 26% and 25%, respectively; 3) Compared with BBR and its variants, it achieves over 25% reduction in tail latency while maintaining stable throughput.

2573-301X/25/$31.00 ©2025 IEEE
DOI 10.1109/CBD69312.2025.00020

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 08,2026 at 18:17:30 UTC from IEEE Xplore. Restrictions apply.

Our main contributions are as follows:

- We identify and quantify the residual queue buildup problem in BBR caused by periodic bandwidth overshoot.
- We propose AQDC-BBR, a novel and lightweight extension to BBR that tracks and compensates for "queue debt" by adjusting inflight data, enabling accurate RTT tracking without sacrificing throughput.
- We implement AQDC-BBR in ns-3 and rstun, and conduct comprehensive simulation tests. Results show that AQDC-BBR reduces tail latency by 25%–30% compared with other schemes while maintaining consistently high throughput.

In the following sections, Section 2 discusses the design motivation of AQDC-BBR. Section 3 describes its design and advantages. Section 4 presents the simulations and experiments related to AQDC-BBR. Finally, Section 5 summarizes the key findings of this paper.

II. Background and Motivation

A. BBR Congestion Control: Mechanism and Potential Limitations

BBR (Bottleneck Bandwidth and Round-Trip Time) is a model-driven congestion control algorithm. Its core idea is to continuously estimate the maximum bandwidth of the link (maxBw) and the minimum round-trip time (minRTT), and use their product as an estimate of the bandwidth–delay product (BDP) to regulate the congestion window and the sending rate, maintain the amount of inflight data in the link at around 1×BDP.

$$\text{inflight}_{\text{target}} = \text{maxBw} \cdot \text{minRTT} \tag{1}$$

Unlike traditional loss-based algorithms such as Cubic and Reno, BBR does not rely on packet loss as a congestion signal. Instead, it periodically cycles through several phases to actively probe the network state, including:

- Startup: rapidly ramping up the sending rate to discover available bandwidth
- Drain: draining the queues accumulated during overshoot
- ProbeBW: continuously probing for additional bandwidth during steady-state operation
- ProbeRTT: periodically reducing the sending rate to remeasure minRTT

During the ProbeBW phase, BBR temporarily increases the sending rate to typically 1.25× maxBw to detect potential residual capacity. After probing, it enters a ProbeDown subphase, reducing the rate to 0.75× maxBw to drain accumulated queue.

While this design intends to prevent persistent queuing, our experimental results in Fig. 1 indicate that each ProbeBW cycle still leaves residual packets in the buffer. In long-lived flows, this residual queue causes the sender's measured RTT to drift upward until the next ProbeRTT phase, highlighting a limitation in BBR's ability to maintain accurate network state under probing.

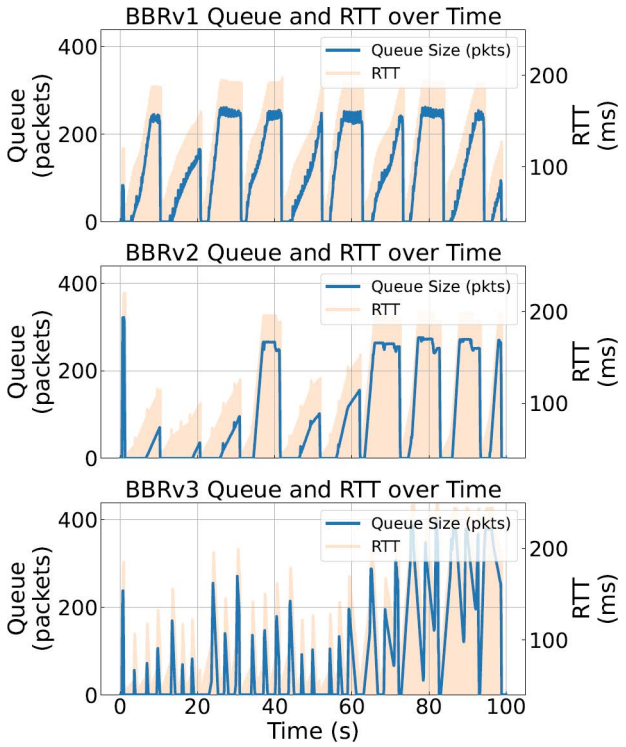


Fig. 1: Queue Buildup and RTT Evolution under BBR Variants.

B. Design Challenges: Latency Control under BBR's Fairness Model

Despite BBR's intention to combine high throughput with low latency, we identify two core limitations that pose design challenges in delay-sensitive scenarios such as video streaming.

Limitation 1: Fairness Requires Queueing, which Inflates Latency. BBR improves bandwidth utilization by periodically probing the network with a higher pacing rate (typically 1.25× maxBw). Whenever a new, higher delivery rate is observed, the congestion window is increased, introducing additional traffic into the network. However, if the probed bandwidth is overestimated or not fully sustained, the resulting queue is only partially drained during subsequent ProbeDown phases. Over successive ProbeBW cycles, these residual queues accumulate, inflating observed RTT and causing the congestion control model to operate above the true BDP.

Challenge 1. To achieve fairness, BBR must interact with bottleneck queues; yet, these queues directly conflict with the goal of low-latency operation. How can we maintain fairness without letting queues degrade delay-sensitive performance?

Limitation 2: ProbeRTT is Rigid and Blind to Queue History. To mitigate queue buildup, BBR periodically enters a ProbeRTT phase, which reduces inflight to a fixed small value (4 packets in BBRv1) for a fixed duration (200ms). While

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 08,2026 at 18:17:30 UTC from IEEE Xplore. Restrictions apply.

this mechanism occasionally resets latency, the sudden rate reduction can cause throughput instability, and the parameters lack principled grounding. Most importantly, ProbeRTT does not consider how much queue debt has actually accumulated; it resets blindly, independent of past behavior.

Challenge 2. This limitation motivates the need for a more responsive ProbeRTT design that accounts for the actual accumulated queue debt, allowing the sender to restore accurate minRTT measurements without unnecessary throughput drops or over-aggressive rate cuts.

Design Insights. From these challenges, we derive three key insights that guide AQDC-BBR:

- Queue buildup is unavoidable and necessary for fair bandwidth convergence under BBR-style probing;
- The additional queue generated in each ProbeUp phase is local and predictable, based on the observed delivery rate increase;
- By proportionally reducing inflight data during ProbeRTT according to accumulated queue debt, we can restore accurate minRTT without aggressive heuristics.

These insights form the foundation of our proposed AQDC-BBR framework, introduced in the next section.

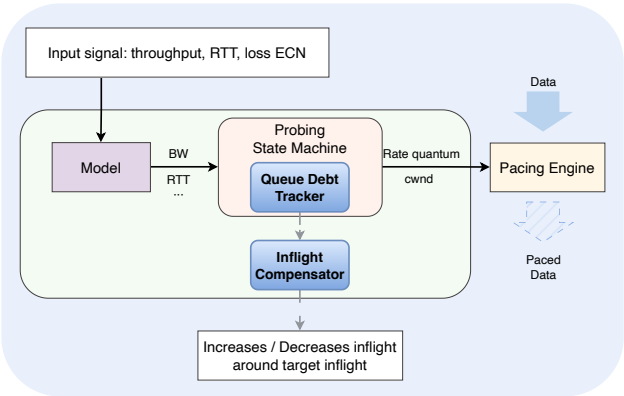


Fig. 2: AQDC-BBR Control Loop with Queue Debt Compensation Modules.

III. AQDC-BBR OVERVIEW

We design the Active Queue Debt Compensation (AQDC) mechanism, illustrated in Fig. 2, to augment the original BBR framework with queue-aware responsiveness. AQDC introduces two lightweight modules that track probe-induced queue buildup and proportionally adjust inflight data, enabling more accurate RTT estimation and timely congestion response. These modules are seamlessly integrated into the BBR control loop in a bypass fashion, preserving the original RTT and bandwidth sampling mechanisms while ensuring high compatibility and minimal deployment overhead.The framework consists of two parts:

- **Queue Debt Tracker:** Keeps track of how much extra data each bandwidth probe injects into the network. It adds up these "queue debts" so the algorithm knows how much backlog has been built up over time.
- **Inflight Compensator:** Uses the tracked queue debt to adjust the amount of data in flight. By temporarily sending less, it helps drain the queues and brings RTT back closer to the real minimum, without sudden rate drops or hurting throughput.

A comprehensive breakdown of AQDC's components is presented in Section IV.

IV. AQDC-BBR DESIGN

Building on the AQDC-BBR overview, we now detail the queue debt modeling, compensation logic, and implementation strategy.

A. Queue Debt Modeling

As BBR periodically increases pacing rate during ProbeBW, it may inject more data than the link can handle. When this increase is not fully absorbed by available bandwidth, packets accumulate in the queue. We formalize this phenomenon as queue debt, which quantifies the mismatch between aggressive probing and true bandwidth availability.

During each ProbeBW bandwidth probe, if the estimated bandwidth increases from bw_i to bw_{i+1}, the additional data injected over a minRTT interval constitutes a queue debt:

$$B_i = (bw_{i+1} - bw_i) \cdot \text{minRTT} \tag{2}$$

Over multiple probing cycles, these individual contributions accumulate, forming a total queue debt:

$$B_{\text{total}} = \sum_j B_j \tag{3}$$

This accumulated backlog occupies link buffers and effectively increases the network delay observed by the sender. As a result, the measured RTT rises proportionally with the total queue debt:

$$\begin{aligned} RTT_{\text{observed}} &= \text{minRTT} + \frac{B_{\text{total}}}{C} \\ &= \text{minRTT} + \frac{1}{C}\sum_{j=1}^{n}(bw_{j+1} - bw_j) \cdot \text{minRTT} \end{aligned} \tag{4}$$

Assuming each probing step increases bandwidth by a fraction α:

$$RTT_{\text{observed}} \approx \text{minRTT} + \frac{\text{minRTT} \cdot bw_0}{C}\left((1+\alpha)^n - 1\right) \tag{5}$$

As shown in (5), RTT grows exponentially with each cycle if the sender continues probing without compensation.This is consistent with our experimental observations in Fig. 1, where long-lived BBR flows gradually accumulate queue even in the absence of competing traffic.

To reconcile this, ProbeRTT should not merely re-sample minRTT, but actively repay the queue debt accumulated during

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 08,2026 at 18:17:30 UTC from IEEE Xplore. Restrictions apply.

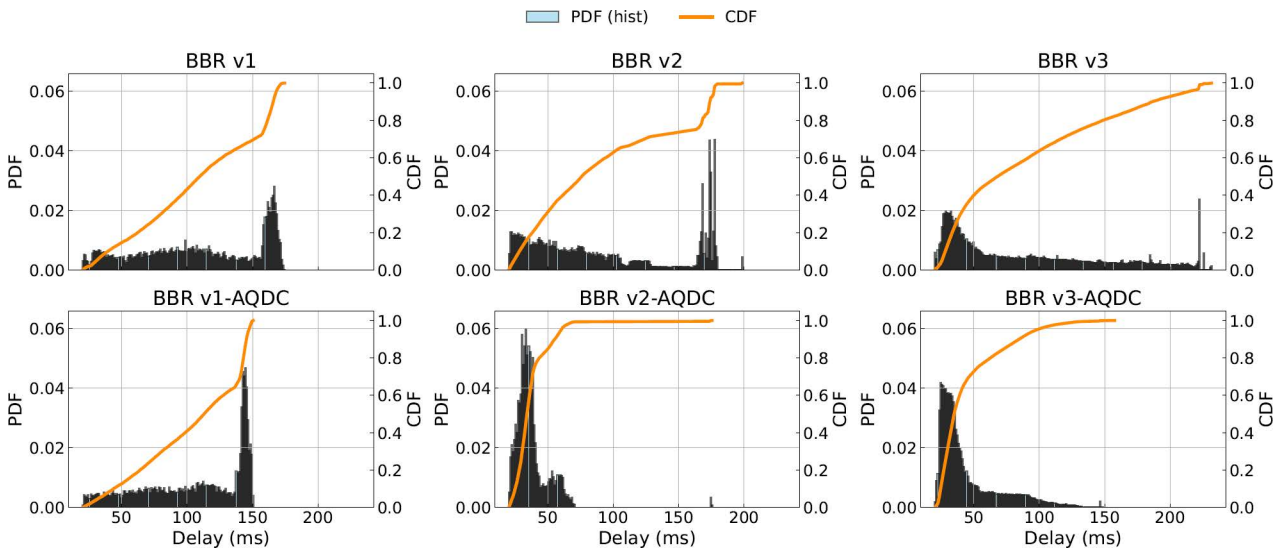


Fig. 3: RTT Distribution (CDF and PDF) for BBR and AQDC-BBR Variants.

past ProbeBW cycles. Concretely, if a flow observes a new delivery rate bw_2 after a ProbeUp phase (up from bw_1 after the previous ProbeRTT), the additional queue injected can be explicitly quantified as:

$$B_1 = (bw_2 - bw_1) \cdot \text{minRTT} \tag{6}$$

Over multiple cycles before the next ProbeRTT, these contributions accumulate:

$$B_{\text{total}} = \sum_i B_i \tag{7}$$

B_{total} reflects the buffer space the flow has implicitly claimed to converge to a fair share. Reducing inflight by B_{total} during ProbeRTT allows the link to drain, restoring the RTT to its true minimum.

The effective throughput under accumulated queue debt is:

$$\text{Throughput} \approx \min\left(\text{BDP}, \frac{\text{inflight}}{RTT_{\text{observed}}}\right) \tag{8}$$

Relationship captured by (8) explains the delay-throughput tradeoff: as queue-induced RTT inflates, throughput may decline unless the inflight is adjusted accordingly.

By stabilizing RTT while maintaining high throughput, AQDC benefits delay-sensitive applications such as real-time video streaming.

B. Queue Debt Compensation Logic

AQDC repays accumulated queue debt during BBR's low-rate phases. The target inflight is adjusted as:

$$\text{Inflight}_{\text{target}} = \text{BDP} - B_{\text{total}} \tag{9}$$

To integrate AQDC into the BBR control loop, we introduce only two lightweight state variables and modify three functions in the core loop. The entire logic is summarized below. The algorithm is minimally invasive, requiring only 30 lines of additional code and no new control states, making it practical for real deployment. Note that our design does not alter RTT or bandwidth estimation modules, preserving BBR's core structure.

Algorithm 1 Queue Debt Compensation in BBR

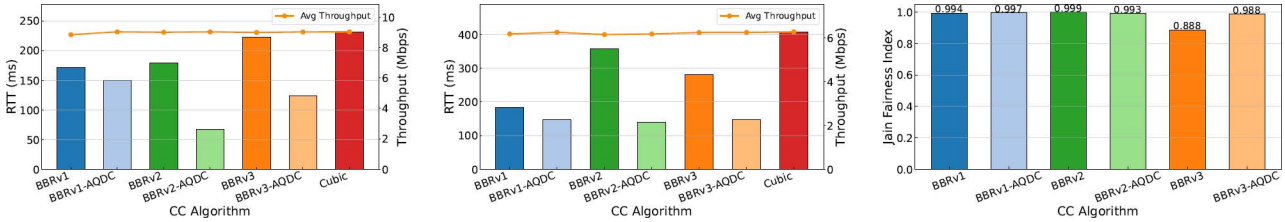
State Variables: $bw_prev \leftarrow 0$, $queue_debt_total \leftarrow 0$
function UPDATEBWESTIMATE(bw_{new})
  if $bw_{new} > bw_prev$ **then**
    $\Delta B \leftarrow (bw_{new} - bw_prev) \cdot minRTT$
    $queue_debt_total \leftarrow queue_debt_total + \Delta B$
  $bw_prev \leftarrow bw_{new}$
function SETINFLIGHTTARGET()
  $BDP \leftarrow bw_{current} \cdot minRTT$
  $inflight_target \leftarrow BDP - queue_debt_total$
function ONPROBERTTSTART()
  ▷ Drain excess queue
  Call SETINFLIGHTTARGET
function ONPROBERTTEND()
  $queue_debt_total \leftarrow 0$

V. Evaluation

A. Methodology and Experimental Setup

We evaluate the proposed AQDC-BBR mechanism through extensive simulations and real-world experiments, focusing on its ability to control queue buildup, stabilize RTT, and deliver low-latency performance in video streaming. We compare AQDC-BBR against BBRv1, BBRv2, and representative congestion control algorithms including Cubic, PCC-Vivace,

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 08,2026 at 18:17:30 UTC from IEEE Xplore. Restrictions apply.

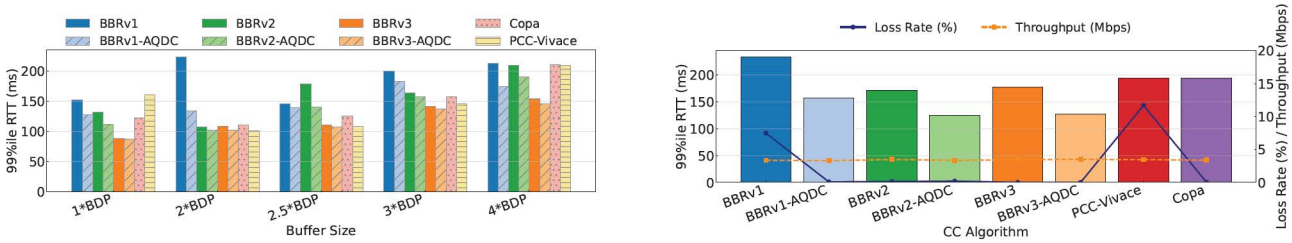


(a) Throughput and Tail RTT under deep buffer Conditions

(b) Throughput and Tail RTT under Time-Varying Bandwidth Conditions

(c) Jain's fairness index comparisons under muti-flow Conditions

Fig. 4: NS-3 Simulation-Based Evaluation of AQDC-BBR.



(a) Effect of Buffering on Tail Latency

(b) P99 RTT and Packet Loss

Fig. 5: Mininet Simulation-Based Evaluation of AQDC-BBR.

and Copa, demonstrating its effectiveness across diverse network conditions. To ensure both protocol-level precision and application-level relevance, we adopt a dual-platform evaluation framework combining NS-3 simulations with Mininet-based video streaming experiments.

NS-3 Network Simulations. NS-3 allows precise control over network parameters, including link bandwidth, propagation delay, and buffer size. We deploy a dumbbell topology with a bottleneck link of 20Mbps and peripheral links of 100Mbps. Many scenarios are considered to evaluate fairness and queue dynamics. Our baselines include BBR v1, v2, v3, Cubic and their AQDC-enhanced variants.

Mininet Video Streaming Experiments. To complement packet-level simulations with application-level performance, we implement QUIC-based video streaming in Mininet using a Rust QUIC stack and ZLMediaKit as the video server. The same set of congestion control algorithms: BBR variants, AQDC-BBR, PCC, and Copa are evaluated.

B. NS-3 Simulation Results

Scenario 1: Tail Latency and RTT Distributions.

We evaluate AQDC-BBR in single-flow scenarios to investigate its impact on latency characteristics across different BBR versions. Specifically, we measure the RTT distributions for BBRv1, BBRv2, and BBRv3, along with their AQDC-enhanced variants. Each flow runs over a fixed-capacity bottleneck with controlled buffer settings, allowing us to isolate protocol-level behaviors. Figure 3 presents both the CDF and PDF of RTT measurements. The results show that AQDC-BBR consistently reduces tail latency across all versions, with 99th percentile RTT improvements ranging from 25% to 40% compared to their unmodified counterparts. This effect is particularly prominent in BBRv2 and BBRv3, where the original design tends to accumulate residual queues during repeated probing cycles.

Scenario 2: Impact of Deep Buffers. We evaluate the performance of AQDC-BBR under deep-buffered conditions, a common scenario in real-world deployments where buffer sizes at bottleneck routers exceed the bandwidth-delay product. In this experiment, we set the bottleneck queue to 4×BDP and observe how different congestion control variants behave in terms of both delay and throughput. Figure 4a illustrates the measured 99th percentile RTT and throughput for BBRv1–v3 and their AQDC-enhanced versions. Standard BBR variants suffer from significant RTT inflation due to the accumulation of persistent queues over multiple probing cycles. Even though throughput remains relatively high, the inflated delay makes these versions unsuitable for latency-sensitive applications. In contrast, AQDC-BBR successfully mitigates excessive queue buildup. As a result, it maintains consistently low tail latency without compromising throughput performance.

Scenario 3: Dynamic Bottleneck Scenario. We evaluate AQDC-BBR under a time-varying bottleneck link, where capacity alternates between 20Mbps and 10Mbps across multiple intervals, mimicking mobile or congested networks. Figure 4b shows throughput and P99 RTT over time. Standard BBR variants respond slowly to bandwidth changes, leading to transient queue buildup and prolonged RTT inflation. In contrast, AQDC-BBR responds quickly to bandwidth changes, consistently maintaining lower tail latency while preserving throughput. This demonstrates AQDC-BBR's adaptability and

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 08,2026 at 18:17:30 UTC from IEEE Xplore. Restrictions apply.

queue-aware design under dynamic network conditions.

Scenario 4: Multi-Flow Fairness Scenario. We evaluate AQDC-BBR in a multi-flow scenario where several flows share the same bottleneck link. Figure 4c shows the Jain's fairness index [21] achieved by standard BBR variants and their AQDC-enhanced versions when multiple flows share the same bottleneck. While BBRv3 exhibits relatively poor fairness in multi-flow scenarios, AQDC-BBR improves its fairness index by 11%, indicating that applying AQDC does not compromise fairness: all flows converge to similar bandwidth shares while benefiting from reduced tail latency. This demonstrates that explicit queue debt compensation can improve delay performance without penalizing bandwidth allocation among competing flows.

C. Video Streaming Scenario Results

Scenario 5: Sensitivity to BDP in Video Streaming. We evaluate AQDC-BBR in video streaming scenarios under varying bandwidth-delay products (0.1, 1, 2, 4×BDP) to test its robustness across different network capacities and buffer regimes. Figure 5a reports the 99th percentile RTT under each setting. AQDC-BBR consistently outperforms unmodified BBR variants, achieving over 25% reduction in tail latency while maintaining stable throughput. Compared to Copa and PCC, AQDC-BBR reduces tail latency by 26% and 25%, respectively, demonstrating its effectiveness and adaptability across diverse BDP conditions without requiring parameter tuning.

Scenario 6: Performance under Tail Latency and Loss in Video Streaming. We evaluate AQDC-BBR in realistic video streaming scenarios using Mininet and ZLMediaKit. Figure 5b presents the 99th percentile RTT and packet loss rates observed in video streaming experiments. AQDC-BBR achieves significantly lower tail latency, reducing P99 RTT by at least 25% compared to standard BBR as well as other algorithms such as PCC and Copa, while maintaining comparable or slightly improved packet delivery rates. These results indicate that AQDC-BBR effectively stabilizes delay-sensitive flows in practical video streaming scenarios.

VI. Conclusion

In this paper, We identify key limitations in BBR: fairness requires interacting with queues, which inflates latency, and ProbeRTT is rigid, ignoring accumulated queue debt. To address this, we propose AQDC-BBR, which tracks excess queue during bandwidth probes and proportionally reduces inflight during ProbeRTT. This restores accurate minRTT without aggressive rate cuts, preserving fairness while reducing tail latency. Extensive NS-3 and Mininet evaluations show AQDC-BBR reduces P99 RTT by 25%–40% across single- and multi-flow scenarios, deep buffers, and dynamic bandwidth conditions. In video streaming experiments, it lowers latency while maintaining comparable or improved packet delivery rates. AQDC-BBR thus offers a simple, effective approach to reconciling fairness and low-latency goals in model-based congestion control.

References

[1] Xie, Xiufeng, Xinyu Zhang, "Poi360: Panoramic mobile video telephony over lte cellular networks." Proceedings of the 13th International Conference on emerging Networking EXperiments and Technologies. 2017.

[2] Wang, Haiyang, "Toward cloud-based distributed interactive applications: measurement, modeling, and analysis." IEEE/ACM Transactions on Networking 26.1 (2017): 3-16.

[3] Ha, Sangtae, Injong Rhee, Lisong Xu, "CUBIC: a new TCP-friendly high-speed TCP variant." ACM SIGOPS operating systems review 42.5 (2008): 64-74.

[4] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobson, "Bbr: Congestion-based congestion control: Measuring bottleneck bandwidth and round-trip propagation time," Queue, vol. 14, no. 5, pp. 20–53, 2016.

[5] Kleinrock, Leonard. "Power and deterministic rules of thumb for probabilistic problems in computer communications." ICC 1979; International Conference on Communications, Volume 3. Vol. 3. 1979.

[6] Brakmo, Lawrence S., Sean W. O'malley, and Larry L. Peterson. "TCP Vegas: New techniques for congestion detection and avoidance." Proceedings of the conference on Communications architectures, protocols and applications. 1994.

[7] Arun, Venkat, and Hari Balakrishnan, "Copa: Practical Delay-Based congestion control for the internet." 15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18). 2018.

[8] Dong, M., Li, Q., Zarchy, D., Godfrey, P. B., Schapira, M, "PCC: Re-architecting congestion control for consistent high performance." 12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15). 2015.

[9] Y. Cao, A. Jain, K. Sharma, Balasubramanian, Gandhi, "When to use and when not to use BBR: An empirical analysis and evaluation study." Proceedings of the Internet Measurement Conference. 2019.

[10] D. Scholz, B. Jaeger, L. Schwaighofer, D. Raumer, F. Geyer, G. Carle, "Towards a deeper understanding of TCP BBR congestion control." 2018 IFIP networking conference (IFIP networking) and workshops. IEEE, 2018.

[11] Hock, Mario, Roland Bless, Martina Zitterbart. "Experimental evaluation of BBR congestion control." 2017 IEEE 25th international conference on network protocols (ICNP). IEEE, 2017.

[12] N. Cardwell, Y. Cheng, S. H. Yeganeh, I. Swett, V. Vasiliev, P. Jha, Y. Seung, M. Mathis, V. Jacobson, "BBR v2: a model-based congestion control IETF 105 update." Presentation at IETF105 (2019).

[13] M. Alizadeh, A. Greenberg, D. Maltz, J. Padhye, P. Patel, B. Prabhakar, S. Sengupta, M. Sridharan, "Data center tcp (dctcp)." Proceedings of the ACM SIGCOMM 2010 Conference. 2010.

[14] A. Nandagiri, M. P. Tahiliani, V. Misra, K. K. Ramakrishnan, "BBRvl vs BBRv2: Examining performance differences through experimental evaluation." 2020 IEEE International Symposium on Local and Metropolitan Area Networks (LANMAN. IEEE, 2020.

[15] J. Gomez, E. Kfoury, J. Crichigno, E. Bou-Harb, G. Srivastava, "A performance evaluation of TCP BBRv2 alpha." 2020 43rd International Conference on Telecommunications and Signal Processing (TSP). IEEE, 2020.

[16] N. Cardwell, Y. Cheng , K. Yang, "BBRv3: Algorithm bug fixes and public internet deployment." Proceedings of the IETF-117. Internet Engineering Task Force, San Francisco, CA, USA 26 (2023).

[17] N. Cardwell, Neal, Y. Cheng , K. Yang, "BBR congestion control: Fundamentals and updates.", 2023.

[18] P. Goyal, A. Narayan, F. Cangialosi, D. Raghavan, S. Narayana, M. Alizadeh, and H. Balakrishnan, "Elasticity detection: A building block for delay-sensitive congestion control." Proceedings of the 2018 Applied Networking Research Workshop (ANRW '18), 2018.

[19] Riley, George F., and Thomas R. Henderson. "The ns-3 network simulator." Modeling and tools for network simulation. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010. 15-34.

[20] Mininet: An Instant Virtual Network on Your Laptop (or Other PC) - Mininet. URL http://mininet.org/

[21] Floyd, Sally. Metrics for the evaluation of congestion control mechanisms. No. rfc5166. 2008.

[22] A. Mishra, X. Sun, A. Jain, S. Pande, R. Joshi, and B. Leong, "The great internet tcp congestion control census," Proc. ACM Meas. Anal. Comput. Syst., vol. 3, no. 3, Dec. 2019.

Authorized licensed use limited to: Universitaetsbibliothek der RWTH Aachen. Downloaded on April 08,2026 at 18:17:30 UTC from IEEE Xplore. Restrictions apply.